Lecture L03

*Blind review — please score this lecture on row **L03** in the Excel form (1–10 per dimension).*

Learning Objectives
By the end of this lecture, you will be able to:
1. **Identify** whether a given relation schema violates 1NF, 2NF, 3NF, or BCNF by applying formal definitions.
2. **Explain** how partial and transitive dependencies lead to insertion, deletion, and update anomalies.
3. **Apply** decomposition techniques to convert a schema into 2NF, 3NF, and BCNF, ensuring lossless join and dependency preservation where possible.
4. **Analyze** a database schema to evaluate trade-offs among normalization levels, including redundancy, anomaly risk, and constraint enforcement.
5. **Design** a normalized schema from a real-world scenario using functional dependencies and candidate keys.

Prerequisites: Functional Dependencies and Keys

Before proceeding, we establish foundational definitions.

A **functional dependency (FD)** $X \to Y$ holds in a relation if, for any two tuples, equality on all attributes in X implies equality on all attributes in Y.
For example, if `EmployeeID → Department`, then no two tuples with the same `EmployeeID` can differ in `Department`.

Think of an FD as a *data promise*: whenever two rows agree on X, they must also agree on Y. Break that promise, and your data lies.

A **candidate key** is a minimal set of attributes that functionally determines every other attribute in the relation. It satisfies two properties:
- **Uniqueness**: No two distinct tuples have identical values on the key.
- **Minimality**: No proper subset of the key functionally determines all attributes.

Here's a subtle but critical insight: a candidate key is not just an identifier—it's the *anchor* of truth for the entire row. Every other attribute depends on it, directly or indirectly. Lose that anchor, and the schema floats on assumptions.

With these tools, we now examine a motivating example—one that hides a paradox.

Motivating Example: The Paradox of the Missing Instructor

Imagine a university database tracking students, courses, and instructors. Suppose we begin with a single relation:

```
TeachRecord(Student, Course, Instructor)
```

Sample instance:
Student	Course	Instructor
Alice	Math101	Dr. Smith

Bob	CS101	Dr. Lee
Charlie	Math101	Dr. Smith
Alice	CS101	Dr. Lee

At first glance, this seems fine. But ask yourself: *Can the database answer the question "Who teaches CS101?" without relying on student enrollment?*

Suppose Dr. Lee teaches only CS101. Now consider:
- **Insertion anomaly**: You cannot record that Dr. Lee teaches CS101 unless at least one student is enrolled.
- **Deletion anomaly**: If all students drop CS101, the fact that Dr. Lee teaches it vanishes.
- **Update anomaly**: If Dr. Lee changes her name, and the update applies to only some rows, the database reports both "Dr. Lee" and "Dr. Lopez" for the same course.

The paradox? *The database claims to store knowledge about teaching, but its ability to preserve that knowledge depends on unrelated data—student enrollment.*

This is not a bug. It's a design flaw: the schema conflates *who is enrolled* with *who teaches what.* The cost? Fragile data semantics.

> **Try this yourself**: Sketch a scenario where updating one fact (e.g., instructor name) requires changing multiple rows. Why does this happen? What dependency is being violated?

Having seen the consequences, we now formalize a path to robust design.

---

1. First Normal Form (1NF): The Atomicity Boundary

Normalization begins with **First Normal Form (1NF)**, which defines the smallest unit of data the database can reason about.

Formal Definition A relation is in **1NF** if and only if:
- Every attribute contains atomic (indivisible) values.
- There are no repeating groups (e.g., `Course1`, `Course2`, `Course3` columns).

This ensures that each cell holds exactly one value, enabling precise querying and integrity enforcement.

Analogy: The Periodic Table of Data Just as chemistry relies on atoms as indivisible building blocks, databases rely on atomic values. You can't split a proton in chemical equations; you can't split a comma-separated list in reliable queries. If `Tags = "bug, ui"` lives in one cell, the database treats it as a *single molecule*, not two elements. You can't index "ui" alone. You can't enforce constraints on individual tags. You've left the domain of relational algebra.

Worked Example Suppose we define:

```
StudentCourses(StudentID, Name, Courses)
```

With sample data:
StudentID	Name	Courses
———	——	————-
1	Alice	{CS101, CS102}

Here, `Courses` contains a set — a non-atomic value — violating 1NF.

To satisfy 1NF, decompose into flat, single-valued tuples:

`Enrollment(StudentID, CourseID)`

StudentID	CourseID
1	CS101
1	CS102

Each row now represents a single enrollment, and all attributes are atomic.

> **Common Misconception**: "1NF requires unique rows."
> **Correction**: 1NF does not mandate uniqueness. Duplicate rows are permitted under 1NF but typically disallowed via primary key constraints to prevent redundancy.

> **Formative Check 2**: Is the following table in 1NF?
> | ID | Tags |
> |—-|————-|
> | 1 | "bug, ui" |
> **Answer**: No. The `Tags` attribute contains a concatenated string, which is not atomic. To normalize, create a separate relation: `IssueTags(ID, Tag)`.

Now that we've defined the atomic unit, we turn to redundancy—not from structure, but from logic.

---

2. Second Normal Form (2NF): The Cost of Partial Truths

After achieving 1NF, we target **partial dependencies**—situations where a non-key attribute depends on only part of a composite key.

Formal Definition A relation is in **Second Normal Form (2NF)** if:
- It is in 1NF, and
- Every **non-prime attribute** is **fully functionally dependent** on every candidate key.

An attribute is **prime** if it is part of any candidate key; otherwise, it is **non-prime**.
A **partial dependency** occurs when a non-prime attribute depends on a proper subset of a candidate key.

Analogy: The Library Book Barcode Imagine a library where each book has a unique barcode (composite key: `ShelfNumber, PositionOnShelf`). Now suppose the book's genre depends only on `ShelfNumber`. That means:
- Every book on shelf 5 is sci-fi.
- But the database stores "sci-fi" in every row for shelf 5.

If you move a book to shelf 6, you must update its genre—even though genre is logically tied to shelf, not the book. The dependency is partial, and the schema pays the price in redundancy.

The flaw? *Partial keys should not govern full truths.*

Worked Example Consider:

`OrderDetails(OrderID, ProductID, ProductName, Quantity)`

Assume the following:
- Functional dependency: ProductID → ProductName
- Candidate key: {OrderID, ProductID}

Now evaluate ProductName. It depends only on ProductID, a subset of the key. Since ProductName is non-prime and partially dependent on the key, the schema violates 2NF.

To resolve this, decompose into two relations:
1. `OrderItems(OrderID, ProductID, Quantity)` — key: {OrderID, ProductID}
2. `Products(ProductID, ProductName)` — key: {ProductID}

In `OrderItems`, all non-prime attributes (e.g., `Quantity`) depend fully on the entire key. In `Products`, `ProductName` depends on the full key `ProductID`. The decomposition eliminates partial dependencies.

> **Try this yourself**: Suppose `Enrollment(StudentID, CourseID, CourseName)` with key {StudentID, CourseID} and FD CourseID → CourseName. Why does this violate 2NF? What happens if two students enroll in the same course? How many times is `CourseName` repeated?
>
> **Formative Check 3**: Decompose the above schema into 2NF. Does your decomposition preserve all functional dependencies? Can you reconstruct the original data without loss?

With partial dependencies resolved, we now face a deeper problem—one where truth travels through chains.

---

3. Third Normal Form (3NF): Breaking the Dependency Chain

Even in 2NF, anomalies persist when non-key attributes determine other non-key attributes—**transitive dependencies**.

Formal Definition A relation is in **Third Normal Form (3NF)** if:
- It is in 2NF, and
- For every nontrivial functional dependency $X \to Y$, **at least one** of the following holds:
1. X is a **superkey**, or
2. Y is a **prime attribute** (i.e., part of some candidate key).

This prevents non-prime attributes from being determined by non-superkeys.

Analogy: The Game of Telephone Imagine a chain:
`EmployeeID → Department → Manager`

When Alice changes departments, her manager changes—not because she chose a new manager, but because the department carries that fact. Now suppose the manager of Data Science changes. You must update *every employee* in that department. Miss one? Now Alice still reports to the old manager.

This is the *data version of telephone*: information passed through intermediaries distorts. 3NF breaks the chain by ensuring that only authoritative sources (superkeys) can determine facts.

Worked Example Consider:

`Employee(EmployeeID, Department, Manager)`

With functional dependencies:
- EmployeeID → Department
- Department → Manager

Candidate key: {EmployeeID}
Non-prime attributes: `Department`, `Manager` (neither is part of any candidate key)

Examine Department → Manager:
- Department is not a superkey.
- `Manager` is not a prime attribute.

Thus, the schema violates 3NF.

To resolve, decompose into:
1. `EmployeeDept(EmployeeID, Department)` — key: {EmployeeID}
2. `DeptManager(Department, Manager)` — key: {Department}

Now, `Manager` is determined directly by `Department`—a superkey in its own table. The chain is broken. Updates to management affect only one row per department.

> **Formative Check 4**: In the decomposed schema, can you still determine Alice's manager? How? What if two departments have the same manager? Is that a problem?
>
> **Try this yourself**: Design a scenario where a transitive dependency causes a deletion anomaly. What is lost when a row is deleted? Could that fact have been preserved?

---

4. Boyce-Codd Normal Form (BCNF): The Gold Standard

3NF handles most cases—but not all. There exists a rare but critical flaw 3NF allows: a dependency where the determinant is not a superkey, even if the dependent attribute is prime.

Formal Definition A relation is in **Boyce-Codd Normal Form (BCNF)** if:
For every nontrivial functional dependency $X \to Y$, X is a **superkey**.

BCNF is stricter than 3NF: it removes the "or Y is prime" loophole. If something determines anything, it must be a superkey.

Analogy: The Passport Control Line Imagine an airport where boarding passes are checked. Two rules:
- Passengers with connecting flights go to Gate A.
- All passengers with boarding pass type "International" go to Gate A.

Suppose ConnectingFlight → Gate and PassportType → Gate. But neither attribute is a full identifier for the passenger.

Now, if Gate A closes, you must update *every row* where either condition holds. Worse: if a passenger has both, you've stored the same gate twice—redundancy from overlapping determinants.

BCNF says: *Only full identities (superkeys) get to make binding decisions.* Otherwise, split the logic.

Worked Example Consider a relation:

`CourseInstructor(Course, Instructor, Department)`

With FDs:
- Instructor → Department (each instructor belongs to one department)
- Course → Instructor (each course has one instructor)

Candidate keys: {Course}, {Instructor, Department}? Wait—check uniqueness.

Actually, if Course → Instructor and Instructor → Department, then Course → Department by transitivity. So Course determines all attributes—candidate key: {Course}.

Now examine Instructor → Department:
- Instructor is not a superkey (different courses could have same instructor? No—Course → Instructor, but not vice versa).
- So Instructor does not determine Course, hence not a superkey.

Thus, Instructor → Department violates BCNF.

Decompose into:
1. `CourseInstructor(Course, Instructor)` — key: {Course}
2. `InstructorDept(Instructor, Department)` — key: {Instructor}

Now all determinants are superkeys. The schema is in BCNF.

> **Formative Check 5**: Why didn't 3NF catch this? Because `Department` is a prime attribute in some candidate keys? Wait—no. In the original schema, is `Department` part of a candidate key? (Answer: no—only `Course` is a candidate key. `Instructor` is not unique. So actually, this violates 3NF too. But BCNF would catch it even if it didn't.)
>
> **Key Insight**: BCNF is about *determinant authority*. It doesn't care what's being determined—only that the determiner is a superkey. It's the ultimate defense against hidden redundancy.
>
> **Try this yourself**: Can every 3NF schema be converted to BCNF with lossless join? What about dependency preservation? (Hint: No—sometimes you must sacrifice a dependency. Why?)

Summary: The Normalization Lattice

We've climbed a hierarchy:
- **1NF**: Data is atomic—no lists, no groups.
- **2NF**: No partial dependencies—whole key matters.
- **3NF**: No transitive dependencies—no chains of determination.
- **BCNF**: Only superkeys determine—no exceptions.

Each step eliminates a class of anomalies by aligning data structure with logical dependency.

But here's the non-obvious truth: *Normalization is not always optimal.* Sometimes, controlled denormalization improves query performance. The key is knowing the cost: increased redundancy, risk of inconsistency.

> **Final Challenge**: Design a schema for a hospital tracking patients, doctors, and specialties.
> - Each doctor has one specialty.
> - Each patient is assigned one doctor.
> - Some specialties have multiple doctors.
>
> Identify FDs, candidate keys, and normalize to BCNF. Where might you choose to denormalize in practice?

Normalization is not dogma. It is *design reasoning*—a way to make dependencies visible, manageable, and truthful.